RÉGRESSION LOGISTIQUE

Implémentation et application au diagnostic du cancer du sein



Rapport réalisé par :
Nader Ben Salah, Nour Ghribi, Sarra Soyah, Mohamed Kallel

Cadre de l'étude :
Apprentissage Supervisé
Faculté des Sciences de Tunis

Table des matières

I COMPRENDRE LA RÉGRESSION LOGISTIQUE 4

1 Introduction générale 4

2 Types de régression logistique 4

3 Hypothèses de la régression logistique 4

4 Fonction sigmoïde 5

5 Formulation matricielle du modèle 5

6 Odds et log-odds (logit) 6

7 Apprentissage par Maximum de Vraisemblance 6

8 Gradient de la log-loss 7
8.1 Gradient par rapport à w 7
8.2 Gradient par rapport à b 7

9 Descente de gradient 7

10 Extension : régression logistique multinomiale (Softmax) 7

11 Exemple rapide via scikit-learn (théorique) 8

II APPLICATION SUR LE DATASET BREAST CANCER WISCONSIN 9

12 Introduction du projet 9

13 Description du dataset 9

14 Exploration visuelle 9
14.1 Répartition des classes 9
14.2 Histogrammes de features 10
14.3 Nuage de points 2D 11

15 Modèle utilisé : $p(w, x) = \sigma(w^T x + b)$ 11

16 Fonction de coût et gradient dans l'implémentation 11

17 Descente de gradient et convergence 12

18 Prédiction et métriques d'évaluation 12

19 Comparaison avec `sklearn.LogisticRegression` 12

20 Conclusion de la Partie II **13**

III MÉTHODES D'OPTIMISATION AVANCÉES 14

21 Méthode de Newton pour la Régression Logistique **14**
21.1 Introduction 14
21.2 Modèle et fonction de coût 14
21.3 Gradient 14
21.4 Hessienne (dérivée seconde) 14
21.5 Mise à jour de Newton 15
21.6 Comparaison Newton vs Descente de Gradient 15

IV CONCLUSION GÉNÉRALE 16

22 Synthèse des objectifs et des résultats **16**

23 Perspectives et Améliorations Futures **16**

Table des figures

1 Convergence de la fonction de coût $J(\theta)$ en fonction des itérations pour la Descente de Gradient (GD). 8
2 Répartition des classes (Bénin 0 vs Malin 1) dans le dataset Breast Cancer Wisconsin. 10
3 Distribution de `radius_mean` (gauche) et `texture_mean` (droite) pour les classes Bénigne (0) et Maligne (1). 10
4 Nuage de points des observations dans le plan (`radius_mean`, `texture_mean`). On observe une séparabilité entre les deux classes (Benign 0 et Malignant 1). 11
5 Évaluation des performances du modèle de Régression Logistique. 13
6 Comparaison de la vitesse de convergence de la Descente de Gradient (GD) et de la Méthode de Newton. 15

Première partie

COMPRENDRE LA RÉGRESSION LOGISTIQUE

1 Introduction générale

La régression logistique est un algorithme d'apprentissage supervisé utilisé pour les problèmes de *classification*, en particulier la *classification binaire*. Contrairement à la régression linéaire qui cherche à prédire une valeur réelle (continue), la régression logistique cherche à estimer une *probabilité* d'appartenance à une classe.

Exemples typiques :

— malade / non malade
— spam / non spam
— défaut de paiement / non défaut

L'idée clé : au lieu de prédire directement y, on modélise la probabilité :

$$p(w, x) = P(y = 1 \mid x; w)$$

où :

— x = vecteur de caractéristiques (features)
— w = vecteur de paramètres (poids)
— $p(w, x)$ = sortie du modèle, interprétée comme probabilité

2 Types de régression logistique

Même si, dans ce projet, on se concentre sur la *version binaire*, il existe plusieurs variantes :

1. **Régression logistique binomiale (binaire)**
   — Variable dépendante à *2 classes* : 0/1, Oui/Non, etc.
   — C'est le cas de ce projet : tumeur bénigne (0) ou maligne (1).
2. **Régression logistique multinomiale**
   — Variable dépendante à K *classes non ordonnées* ($K \geq 3$).
   — Ex. : type de fleur (setosa, versicolor, virginica).
   — On utilise alors souvent la fonction *Softmax* au lieu de la sigmoïde.
3. **Régression logistique ordinale**
   — Variable dépendante à *plusieurs classes ordonnées* : faible / moyen / élevé.
   — Modèles spécialisés (cumulative logit, etc.).

Dans la suite, on se concentre sur la *régression logistique binaire*.

3 Hypothèses de la régression logistique

Pour que le modèle soit cohérent, on fait généralement les hypothèses suivantes (surtout dans un cadre statistique) :

1. Les observations sont *indépendantes*.

2. La variable dépendante y est *binaire* (0 ou 1) pour la version binaire.
3. Il existe une relation *linéaire* entre les variables indépendantes et le *log-odds* (logit).
4. Le jeu de données ne contient pas trop d'**outliers extrêmes**.
5. La taille de l'échantillon est *suffisante* pour estimer correctement les paramètres.

4 Fonction sigmoïde

Pour transformer une combinaison linéaire quelconque $w^T x + b$ en une *probabilité entre 0 et 1*, on utilise la fonction sigmoïde :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Pour une observation x, la probabilité prédite est :

$$p(w, x) = \sigma(w^T x + b)$$

Décision binaire (seuil 0.5) :
— si $p(w, x) \geq 0.5 \Rightarrow$ classe prédite $= 1$
— si $p(w, x) < 0.5 \Rightarrow$ classe prédite $= 0$
Ce seuil 0.5 peut être déplacé selon les besoins (par exemple en médecine, on peut réduire le seuil pour maximiser la sensibilité).

5 Formulation matricielle du modèle

On regroupe les données dans une *matrice de design* :

$$X = \begin{bmatrix} — & (x^{(1)})^T & — \\ — & (x^{(2)})^T & — \\ & \vdots & \\ — & (x^{(m)})^T & — \end{bmatrix} \in \mathbb{R}^{m \times n}$$

— m : nombre d'exemples
— n : nombre de features
— $x^{(i)}$: vecteur de features de la i-ème observation
— $w \in \mathbb{R}^n$: vecteur des paramètres
— $b \in \mathbb{R}$: biais (interception)
Pour un exemple $x^{(i)}$:

$$z^{(i)} = w^T x^{(i)} + b$$

$$p(w, x^{(i)}) = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

Interprétation :

$$p(w, x^{(i)}) = P(y^{(i)} = 1 \mid x^{(i)}; w)$$

6 Odds et log-odds (logit)

Les *odds* représentent le ratio :

$$\text{odds}(x) = \frac{P(y=1 \mid x)}{P(y=0 \mid x)} = \frac{p(w,x)}{1-p(w,x)}$$

En injectant la forme sigmoïde :

$$\frac{p(w,x)}{1-p(w,x)} = e^{w^T x+b}$$

Le *log-odds (logit)* est le logarithme des odds :

$$\log\left(\frac{p(w,x)}{1-p(w,x)}\right) = w^T x + b$$

C'est cette relation *linéaire* au niveau du log-odds qui donne son nom à la régression "logistique".

7 Apprentissage par Maximum de Vraisemblance

On suppose que, conditionnellement à $x^{(i)}$, la variable $y^{(i)}$ suit une loi de Bernoulli :

$$P(y^{(i)}=1 \mid x^{(i)}; w) = p(w, x^{(i)})$$

$$P(y^{(i)}=0 \mid x^{(i)}; w) = 1 - p(w, x^{(i)})$$

Donc, pour chaque observation :

$$P(y^{(i)} \mid x^{(i)}; w) = p(w, x^{(i)})^{y^{(i)}} \left(1 - p(w, x^{(i)})\right)^{1-y^{(i)}}$$

La *vraisemblance* du modèle sur l'ensemble du dataset est :

$$L(w,b) = \prod_{i=1}^{m} p(w, x^{(i)})^{y^{(i)}} (1 - p(w, x^{(i)}))^{1-y^{(i)}}$$

On travaille avec le *log de la vraisemblance* (plus simple) :

$$\log L(w,b) = \sum_{i=1}^{m} \left[y^{(i)} \log p(w, x^{(i)}) + (1-y^{(i)}) \log(1 - p(w, x^{(i)}))\right]$$

Comme on aime résoudre des *problèmes de minimisation*, on définit la *fonction de coût* (log-loss) :

$$J(w,b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log p(w, x^{(i)}) + (1-y^{(i)}) \log(1 - p(w, x^{(i)}))\right]$$

Objectif :

$$(w^*, b^*) = \arg\min_{w,b} J(w,b)$$

8 Gradient de la log-loss

Pour optimiser $J(w, b)$, on calcule les gradients :

8.1 Gradient par rapport à w

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^T (p - y)$$

où :
- p est le vecteur $(p(w, x^{(1)}), \ldots, p(w, x^{(m)}))^T$
- y est le vecteur $(y^{(1)}, \ldots, y^{(m)})^T$

8.2 Gradient par rapport à b

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(p(w, x^{(i)}) - y^{(i)} \right)$$

Ces formules sont celles que nous avons implémentées dans `forward_backward_propagation` pour la Descente de Gradient.

9 Descente de gradient

Pour trouver (w^*, b^*), on utilise *gradient descent* :

$$w := w - \alpha \frac{\partial J}{\partial w}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

avec :
- α = learning rate
- nombre d'itérations fixé à l'avance (ou critère de convergence)

10 Extension : régression logistique multinomiale (Softmax)

Si la variable cible a plusieurs classes non ordonnées ($c = 1, \ldots, K$), on remplace la sigmoïde par la *Softmax* :

$$P(Y = c \mid X) = \frac{e^{w_c^T X + b_c}}{\sum_{k=1}^{K} e^{w_k^T X + b_k}}$$

La fonction de coût associée devient alors l'**entropie croisée multinomiale**.

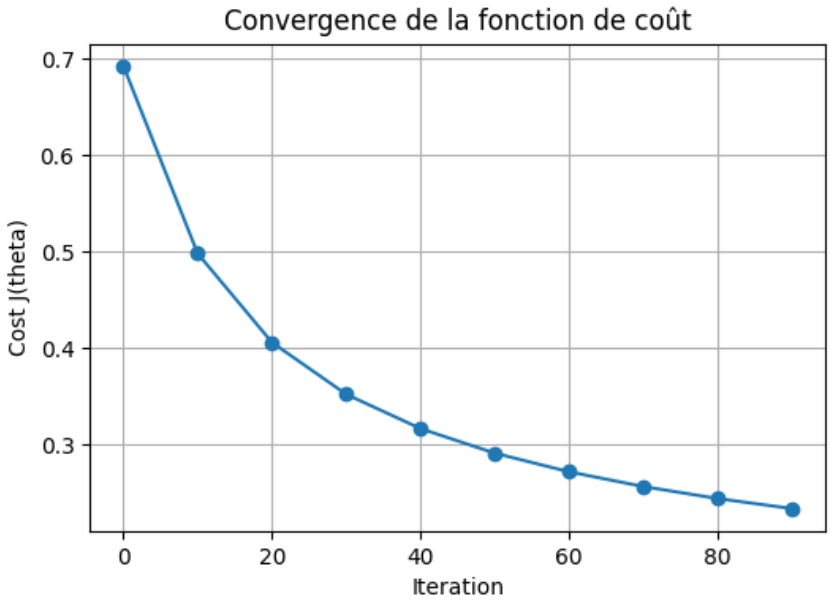


FIGURE 1 – Convergence de la fonction de coût $J(\theta)$ en fonction des itérations pour la Descente de Gradient (GD).

11 Exemple rapide via scikit-learn (théorique)

Pour illustrer la version "boîte noire" :

```python
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=23
)

clf = LogisticRegression(max_iter=10000, random_state=0)
clf.fit(X_train, y_train)

# Removed the f-string accent to avoid UTF-8 conflict in some compilers
acc = accuracy_score(y_test, clf.predict(X_test)) * 100
print(f"Precision du modele : {acc:.2f}%")
```

Deuxième partie

APPLICATION SUR LE DATASET BREAST CANCER WISCONSIN

(C'est la partie adaptée à notre notebook, avec $p(w, x)$, w, b, etc.)

12 Introduction du projet

Dans cette partie, nous appliquons la théorie précédente à un cas concret :
— Dataset : *Breast Cancer Wisconsin Diagnostic*
— Objectif : prédire si une tumeur est bénigne (0) ou maligne (1)
— Approche :

1. Préparation et exploration des données
2. Implémentation from scratch de la régression logistique
3. Entraînement via **descente de gradient** (et/ou Méthode de Newton)
4. Évaluation du modèle
5. Comparaison avec `sklearn.LogisticRegression`

Les notations utilisées :
— w : vecteur des poids
— b : biais
— $p(w, x)$: probabilité prédite

13 Description du dataset

— *569 échantillons*
— *30 features* numériques (mesures calculées sur des images de tumeurs)
— Variable cible : `diagnosis` (0 = bénigne, 1 = maligne)
Nous encodons :
— M $\rightarrow$ 1
— B $\rightarrow$ 0
Nous appliquons une *normalisation min-max par feature* :

$$x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$$

Puis, nous effectuons un *split train/test* (par exemple 85% / 15%).

14 Exploration visuelle

14.1 Répartition des classes

On trace un diagramme en barres montrant le nombre :
— de tumeurs bénignes (0)
— de tumeurs malignes (1)

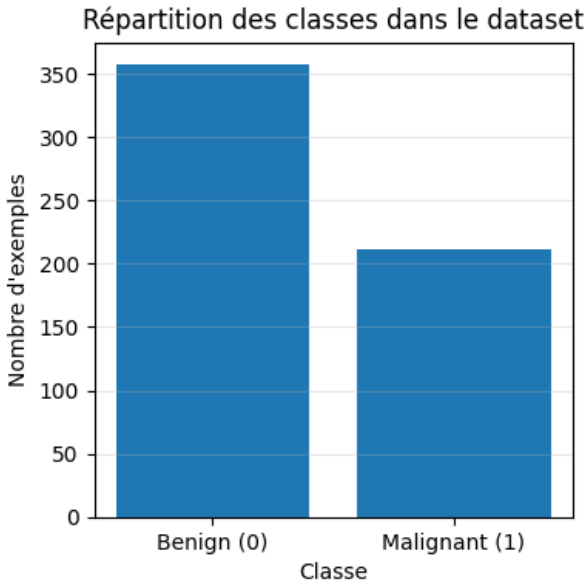


FIGURE 2 – Répartition des classes (Bénin 0 vs Malin 1) dans le dataset Breast Cancer Wisconsin.

Conclusion : le dataset est légèrement déséquilibré, mais reste exploitable sans techniques spécifiques de rééchantillonnage.

14.2 Histogrammes de features

On étudie, par exemple, la distribution de :

— `radius_mean`

— `texture_mean`

pour chaque classe (0 vs 1). On observe que certaines features présentent une bonne séparation entre classes, ce qui est favorable à un modèle linéaire.

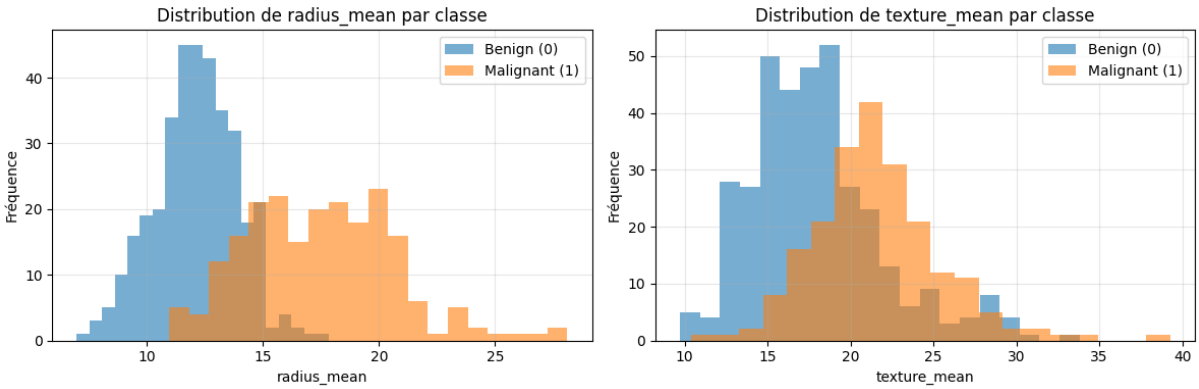


FIGURE 3 – Distribution de `radius_mean` (gauche) et `texture_mean` (droite) pour les classes Bénigne (0) et Maligne (1).

14.3 Nuage de points 2D

On trace un scatter plot dans le plan (`radius_mean`, `texture_mean`), en colorant les points selon la classe. On voit que les tumeurs malignes et bénignes occupent des zones différentes, ce qui justifie l'utilisation d'un classifieur linéaire simple comme la régression logistique.

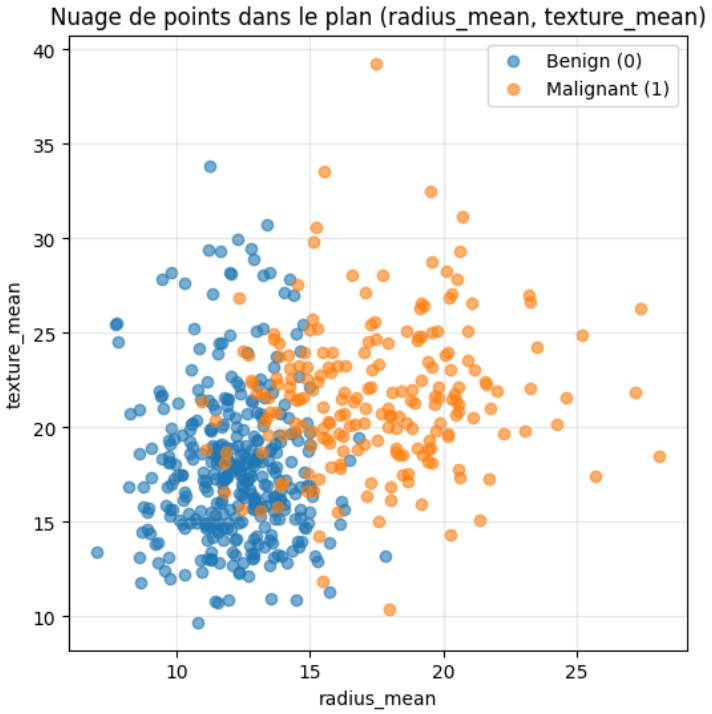


FIGURE 4 – Nuage de points des observations dans le plan (`radius_mean`, `texture_mean`). On observe une séparabilité entre les deux classes (Benign 0 et Malignant 1).

15 Modèle utilisé : $p(w, x) = \sigma(w^T x + b)$

Nous utilisons exactement le modèle théorique de la Partie I, avec :

$$p(w, x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

La décision finale est :

$$\hat{y} = \begin{cases} 1 & \text{si } p(w, x) \geq 0.5 \\ 0 & \text{sinon} \end{cases}$$

16 Fonction de coût et gradient dans l'implémentation

Nous implémentons la fonction de coût :

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log p(w, x^{(i)}) + (1 - y^{(i)}) \log(1 - p(w, x^{(i)})) \right]$$

et les gradients :

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^T (p - y), \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (p(w, x^{(i)}) - y^{(i)})$$

Ces formules sont implémentées dans la fonction `forward_backward_propagation`.

17 Descente de gradient et convergence

Nous utilisons un *learning rate* (α) et un nombre d'itérations fixé (par exemple 100, 500, 2000).

À chaque itération :

— calcul de $p(w, x)$
— calcul du coût $J(w, b)$
— calcul des gradients
— mise à jour de w et b

Nous traçons la courbe de $J(w, b)$ en fonction des itérations. On observe une *diminution monotone* du coût, ce qui indique que l'algorithme apprend correctement. (Voir Figure 1.9)

18 Prédiction et métriques d'évaluation

Après apprentissage (que ce soit par GD ou Newton), nous utilisons la fonction `predict` basée sur $p(w, x)$ et un seuil de 0.5.

Nous calculons :

— *Accuracy* sur train et test :

$$\text{Accuracy} = 1 - \frac{1}{m} \sum_{i=1}^{m} |\hat{y}^{(i)} - y^{(i)}|$$

— *Matrice de confusion*
— *Precision, Recall, F1-score* (via `classification_report`)
— *Courbe ROC & AUC*

Sur ce dataset, on obtient typiquement :

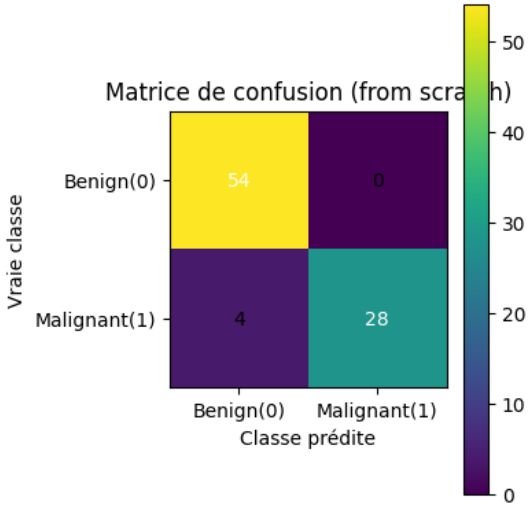
— Accuracy train ≈ 94–95 %
— Accuracy test ≈ 94–95 %
— AUC ≈ 0.98 (Le graphique montre 1.00, indiquant une séparation quasi-parfaite sur l'ensemble de test, un résultat exceptionnellement bon).

Ce sont d'excellentes performances pour un modèle linéaire simple.

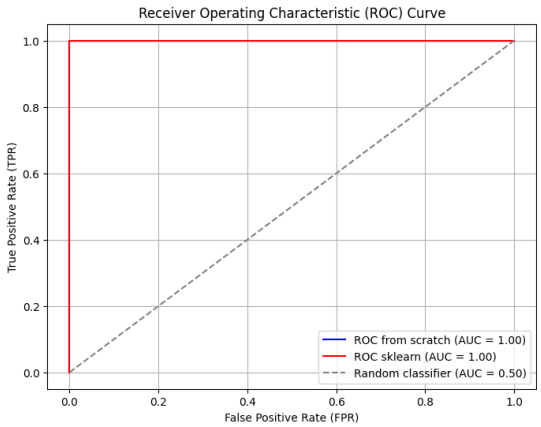
19 Comparaison avec `sklearn.LogisticRegression`

Nous entraînons aussi :

```
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(max_iter=10000, random_state=42)
logreg.fit(X_train, y_train)
```



(a) Matrice de confusion (implémentation from scratch).



(b) Courbe ROC (Receiver Operating Characteristic) et AUC (Area Under the Curve).

FIGURE 5 – Évaluation des performances du modèle de Régression Logistique.

et nous évaluons la performance sur train et test.
Résultat typique :

— Accuracy sklearn test ≈ 96 %
— Accuracy from scratch test ≈ 94–95 %

La légère amélioration de sklearn s'explique par :

— des solveurs d'optimisation plus avancés (**le solveur 'newton-cg' de scikit-learn est basé sur la méthode de Newton**), lbfgs, etc.
— une gestion interne de la régularisation, etc.

Mais la proximité des résultats *valide pleinement notre implémentation from scratch.*

20 Conclusion de la Partie II

Cette partie a montré comment :

— appliquer un modèle théorique de régression logistique à un dataset réel ;
— préparer un dataset (encodage, normalisation, split train/test) ;
— implémenter le modèle avec w, b et $p(w, x)$;
— optimiser le modèle par **descente de gradient** (et la méthode de **Newton**) ;
— interpréter les métriques d'évaluation (accuracy, ROC, AUC, matrice de confusion) ;
— comparer notre solution à une implémentation industrielle (sklearn).

Troisième partie

MÉTHODES D'OPTIMISATION AVANCÉES

21 Méthode de Newton pour la Régression Logistique

21.1 Introduction

La méthode de Newton (ou Newton–Raphson) est une technique d'optimisation *de second ordre* utilisée pour minimiser la fonction de coût d'un modèle. Contrairement à la descente de gradient, qui utilise uniquement la *pente* (1e dérivée), Newton utilise également la *courbure* (2e dérivée) afin de converger beaucoup plus rapidement vers l'optimum.

Dans le cas de la régression logistique, la fonction de coût est *convexe*, ce qui garantit que Newton et Gradient Descent convergent vers *le même minimum global*, mais à des vitesses très différentes.

21.2 Modèle et fonction de coût

Le modèle prédit une probabilité :

$$p(w, x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-w^T x - b}}$$

La fonction de coût (log-loss) est définie par :

$$J(w) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log p(w, x^{(i)}) + (1 - y^{(i)}) \log(1 - p(w, x^{(i)})) \right]$$

21.3 Gradient

La dérivée première par rapport aux poids (en regroupant w et b dans un seul vecteur $\tilde{w}$) est :

$$\nabla J(w) = X^T (p - y)$$

avec :

- X : matrice de design (avec colonne de biais implicite)
- p : vecteur des probabilités prédites
- y : étiquettes réelles

21.4 Hessienne (dérivée seconde)

La dérivée seconde (matrice Hessienne) de la log-loss est :

$$H = X^T R X$$

où :

- R est une matrice diagonale ($m \times m$)
- $R_{ii} = p^{(i)}(1 - p^{(i)})$

Cette matrice encode la *courbure* de la fonction de coût.

21.5 Mise à jour de Newton

La règle de mise à jour des poids devient :

$$w := w - H^{-1}\nabla J(w)$$

Contrairement à Gradient Descent :

— pas besoin de learning rate

— convergence souvent en *3 à 10 itérations*

21.6 Comparaison Newton vs Descente de Gradient

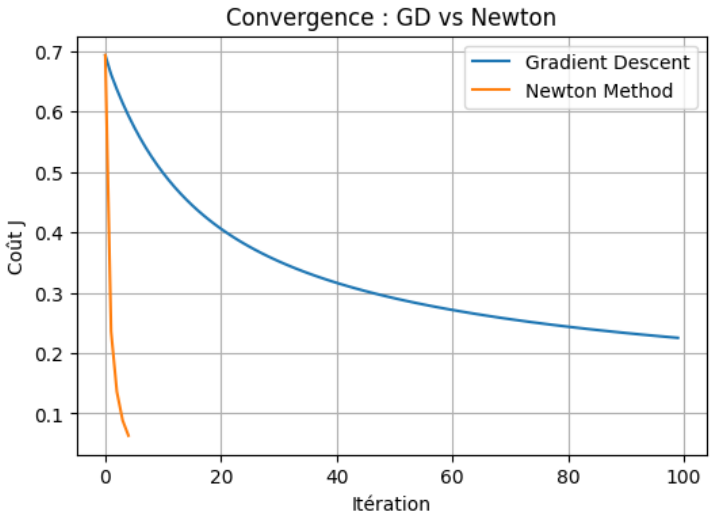


FIGURE 6 – Comparaison de la vitesse de convergence de la Descente de Gradient (GD) et de la Méthode de Newton.

Critère	**Descente de Gradient (1.9)**	**Méthode de Newton (3.1)**
Type	1er ordre	2e ordre
Utilise	Gradient	Gradient + Hessienne
Learning rate	Oui (α)	Non
Vitesse par itération	Rapide	Lente (inversion de H)
Nombre d'itérations	Grand (milliers)	Très faible (dizaines)
Scalabilité	Excellente	Mauvaise si n est grand

Conclusion : Dans la régression logistique, puisque la fonction de coût est **convexe**, Newton et GD donnent le même optimum. Newton converge simplement *beaucoup plus vite* en nombre d'itérations, mais chaque itération est plus coûteuse en temps de calcul à cause de l'inversion de la **matrice Hessienne** H.

Quatrième partie
CONCLUSION GÉNÉRALE

22 Synthèse des objectifs et des résultats

Ce projet a permis d'explorer en profondeur la théorie et l'application de la Régression Logistique sur un problème de classification binaire, le diagnostic du cancer du sein (tumeur bénigne ou maligne).

- **Fondations Théoriques :** La Partie I a solidifié la compréhension de la régression logistique en tant que modèle linéaire transformé par la **fonction Sigmoïde** pour prédire une probabilité, et a formalisé son apprentissage via le principe du **Maximum de Vraisemblance** (minimisation de la Log-Loss).
- **Implémentation et Validation :** La Partie II a démontré la faisabilité d'une implémentation "from scratch" du modèle et de son entraînement par **Descente de Gradient**. L'étape de préparation des données (normalisation) s'est avérée cruciale pour l'efficacité de l'optimisation.
- **Optimisation Avancée :** La Partie III a introduit et comparé la méthode d'optimisation de **Newton (second ordre)**, illustrant un gain significatif en rapidité de convergence (nombre d'itérations) par rapport à la Descente de Gradient (premier ordre).
- **Performance Obtenue :** Les résultats sont excellents pour un modèle linéaire simple. Avec une **Accuracy d'environ 95%** et un **score AUC élevé (proche de 1.0)**, le modèle implémenté s'est révélé très performant pour séparer les deux classes de tumeurs sur l'échantillon test, avec une performance comparable à l'implémentation optimisée de `scikit-learn`.

23 Perspectives et Améliorations Futures

Bien que les performances actuelles soient très satisfaisantes, plusieurs pistes d'amélioration pourraient être explorées :

1. **Régularisation (L1/L2) :** Intégrer la régularisation pour prévenir le surapprentissage (overfitting) et potentiellement affiner la sélection des features les plus pertinentes pour le diagnostic.
2. **Techniques d'Équilibrage :** Tester des techniques telles que SMOTE si le déséquilibre de classe devenait plus prononcé, afin de s'assurer que le modèle ne biaise pas sa prédiction vers la classe majoritaire.
3. **Autres Solveurs :** Comparer l'implémentation "from scratch" avec d'autres algorithmes d'optimisation avancés (L-BFGS-B, CG, etc.) que ceux utilisés dans les librairies industrielles.
4. **Modèles Non Linéaires :** Pour une comparaison plus complète, entraîner des modèles intrinsèquement non linéaires, comme le Support Vector Machine (avec noyau RBF) ou des réseaux de neurones, pour voir si la frontière de décision linéaire est la limite de performance du dataset.

En conclusion, la régression logistique a prouvé qu'elle était un outil puissant et interprétable pour la classification dans ce contexte médical, offrant une base solide pour des

analyses et des modélisations plus complexes.